

Lab Assignment 3

Christos Strydis
Rajendra Bishnoi
Amirreza Movahedin

2026

Contents

1	Introduction	2
2	TVB on GPU Using CuPy	2
3	TVB on GPU Using Numba	3
4	JAX	3

Submission guidelines:

- Hand in the lab report in pdf format with the following name:
Lab3.Firstname.Lastname.Studentnumber (for example: Lab3.Pieter.deBoer.123456)
- Hand in the Python scripts (the **.py** files) as separate, unzipped Python scripts. Do not forget to add meaningful comments to your code. Handing in poorly annotated code will be penalized with one point.
- You must submit your code for exercises **3.1**, **3.2**, **3.4** and **3.5** alongside your report for this lab. Please name your script files with the following convention: **L3-Qx** where x is the exercise number.

1 Introduction

In this third lab assignment, we will go through how to safely parallelize your code using a Graphics Processing Unit (GPU). The theory behind a GPU and how it runs a large number of computations in parallel is explained in class. This lab focuses on how to use the GPU in your Python program.

When you want to use the GPU for your application, you want to transfer the heavy part of your program (the task that requires many computations) from the CPU memory to the GPU memory. We thus need to interact with the GPU (i) to give it the data it needs to process, (ii) to afford it enough time to process, and (iii) to send the processed data back to the CPU.

The interaction with the GPU takes place via CUDA. CUDA is a programming model and Application Programming Interface (API) created by Nvidia for general computing on GPUs. Through Python libraries we have seen before, we can use simple commands that abstract the CUDA programming a lot. Depending on how much control you want over the way the GPU executes part of your program, you can use different libraries. The pattern you will see here is: the more control you want over GPU programming, the more complicated the library will be to use.

2 TVB on GPU Using CuPy

Let us start with the easiest library to use. Before starting with the exercises, take a look at the lecture slides and the documentation of CuPy to familiarize yourself with this library.

Exercise 3.1 (30 points)

You will implement the GPU version of TVB using CuPy based on the provided NumPy-vectorized implementation.

1. Open `tvb_vec.py` script. This is the vectorized version of TVB that you have implemented and seen in the first assignment. Run and record the execution time of this implementation for TVB76, TVB192, and TVB998 for 150ms of simulation. (5 points)
2. Refactor the code in a way that the for-loop that goes over every center at each timestep in the `simulate` function (`for n in range(N):`) is replaced with NumPy array operations inside `F`, `calculate_coupling`, and `step` functions. Run and record the execution time of this implementation for TVB76, TVB192, and TVB998 for 150ms of simulation. (7.5 points)
3. Based on the script you generated in step 2, implement the GPU version of TVB using CuPy. You can refer to CuPy documentation and lecture slides for help. Run and record the execution time of this implementation for TVB76, TVB192, and TVB998 for 150ms of simulation. (7.5 points)
4. Compare the results from all these implementations. How does each one of them scale? How do you explain the results? If we hadn't done the refactorization of step 2, do you think we would have better results for the GPU version or not? (5 points)
5. Profile your GPU code using `nvprof` or `nsys` to generate an execution profile of your application. Apart from printing the profile on the command-line, you can use `nvvp` or `nsys-ui` to get a visual profile. Include a screenshot of the profile in the report. What percentage of the entire runtime is spent on data movement? (5 points)

Exercise 3.2 (20 points)

Implement the high-throughput version of TVB on GPU using CuPy. As a reminder, the high-throughput version of TVB is an implementation where multiple simulations are running independently and in parallel to each other. You have already implemented this version in the previous lab assignment using multiprocessing.

Report the execution time of the code for launching 100, 250 and 500 simulations in parallel, each running the TVB192 dataset for 150ms. Use the same set of weight and distance matrices for all the simulations (in reality different set of parameters are used for different simulations).

Compare the performance of this implementation with the GPU version you implemented in Exercise 3.1 if it was to be executed 100, 250, and 500 times back-to-back. Report the number of simulation iterations (timesteps) per second for each of the implementations. Which one performs better in terms of throughput? Why?

Exercise 3.3 (10 points)

Profile your high-throughput GPU code using `nvprof` or `nsys` to generate an execution profile of your application. Apart from printing the profile on the command-line, you can use `nvvp` or `nsys-ui` to get a visual profile. Include a screenshot of the profile in the report. What percentage of the entire runtime is spent on data movement?

3 TVB on GPU Using Numba

The CuPy module is another way of harnessing the power of GPUs in Python. Another module that enables GPU programming is Numba. We have encountered the Numba module in Lab 1 for compiling our code via the *just-in-time* decorator (`@jit`). This means that part of the Python program was translated to a CPU executable before running the full script, instead of translating Python code line by line (like the Python interpreter does). This led to code-execution speedups. Numba is a JIT compiler for Python code. Its major advantage was its simplicity: by prepending a function call with the JIT decorator was sufficient for compiling the function before execution in the CPU.

Numba, however, can also target the GPU. Just like we could decorate CPU code, we can target GPU code (also known as *GPU kernels*) with `@cuda.jit`. The Numba JIT compiler will compile the piece of decorated code as a GPU executable. Although this sounds easy, the GPU case is somewhat more complicated than the CPU one: Adding this decorator requires specifying some CUDA details. An example was discussed in class, and can be found in the lecture slides. A simpler alternative decorator that Numba offers is `@vectorize` which was also discussed in the lectures. However, we will not cover `@vectorize` decorator in this lab.

Exercise 3.4 (20 points)

Using `@cuda.jit` is similar to coding lower level CUDA program. Implement a single TVB simulation on GPU using this approach. Report the performance of this version on TVB76, TVB192, and TVB998 datasets and compare to the CuPy implementation. (15 points)

Furthermore, *explain* if you were to run multiple simulations at the same time using Numba's `@cuda.jit` on GPU, what would you change in your code. You don't have to implement this version, however you are more than welcome to do so. (5 points)

4 JAX

We introduced and discussed JAX in the lectures. JAX is a high-performance numerical computing library for Python, developed by Google. It provides a familiar NumPy API in the form of `jax.numpy`, making it easy to adopt existing code. Additionally it has JIT compilation (`jax.jit` compiles Python functions into XLA), vectorization of Python functions (`jax.vmap`). Focused more for machine and deep learning applications, it also offers automatic differentiation of functions. JAX codes can easily be executed on CPU, GPU, or Google's TPU. For GPUs, JAX offers the high-level features of CuPy and Numba, without the lower level controls that these frameworks offer. Furthermore, it extends beyond these libraries by allowing arbitrary *PyTrees*, including statically sized dictionaries, lists and dataclasses, as arguments to functions.

Exercise 3.5 (20 points)

We are going to accelerate TVB simulation using JAX on both CPU and GPU. Use the code you generated in second question of Exercise 3.1 in addition to `jax.numpy` and `jax.jit` features to implement the JAX-based version of the TVB algorithm. Run the code on both CPU and GPU for all three datasets for 150ms and report the numbers. How does JAX perform compared to other methods?

- Make sure you enable 64-bit floating point operations by including `jax.config.update("jax_enable_x64", True)`.
- You can choose which backend (CPU or GPU) JAX uses by calling `jax.config.update('jax_platform_name', 'cpu')` or `'gpu', 'tpu'`.

Note that, in contrast to Numba, JAX uses a *tracing* JIT. That means that it executes the function as regular python code, but now with empty placeholder arguments, to discover the compute graph. That means that, unlike in Numba, python statements like for loops and if statements, are executed *before* JIT compilation. Therefore, in general, a for loop leads to an explosion of generated code in JAX, and on top of that, the loop size can only depend on a statically known constant. Instead, you want to either translate these loops to explicit JAX looping constructs like `jax.vmap()` `jax.lax.scan()`, and replace if statements by explicit conditionals like `jax.lax.select()`. Alternatively, for loops, you could only JIT the inside of a looping function, keeping the loop in regular python code. For more information, read JAX - The Sharp Bits. It is sometimes a bit of a puzzle to translate an algorithm into JAX' functional style, but the speedups over other methods are generally worth it.